A sequence diagram showing the CompleteOrder job calling the payments service, getting a 5xx, being retried, getting a 5xx again, then being retried a third time, at which point it succeeds. The job then calls the other two services successfully.

Although the retry mechanism allows the job to succeed eventually, the error catcher still notifies you. Navigate to http://localhost:3001/error-catcher/ui, and you'll see the transient BaseServiceWrapper::HTTPError errors. This means you would be notified about these errors even though no action was required.

This is the core problem to solve when managing Sidekiq job failures. You don't need the error-catching service to notify you about a transient failure, but you *do* need to know about a permanent one.

Prevent Transient Errors from Notifying You

Most error catchers allow you to ignore particular errors. That means you can configure the error catcher to ignore BaseServiceWrapper::HTTPError and you won't get a notification whenever a job failed but was retried. This configuration would apply to more than just jobs, however. If you got a BaseServiceWrapper::HTTPError from, say, a controller, you wouldn't be notified about that either. That's not what you want. You only want to ignore certain errors from Sidekiq jobs. You want to be notified if an error happens outside a Sidekiq job.

A straightforward way to set this up is to create a special exception class that the error catcher will be configured to ignore, thrown only by a job that is experiencing a transient error. This means your job can rescue BaseServiceWrapper::HTTP and re-throw it as this new special exception. Let's call the exception something extremely obvious: IgnorableExceptionSinceSidekiqWillRetry. I won't pretend that's a beautiful class name, but it's certainly self-explanatory.

The code to translate the exception should go in the job code, like so: